Projet Fullstack : co-développement humain-IA

Rapport Final

Auteur : Maxence Moreau
Formation : ENSIM - 4A info FISA
Rapporteur : Emmanuel Blanchard
Date : Mars 2026

Sommaire :

1 - Description du système : contexte, objectifs et fonctionnalités..... 2
1.1 - Contexte et objectif..... 2
1.2 - Fonctionnalités..... 2
1.3 - Expérience utilisateur..... 2
2 - Description du système : résultat, travaux restants et techniques full-stack..... 3
2.1 - Résultat final..... 3
2.2 - Ce qu'il reste à faire..... 3
2.3 - Techniques full-stack pertinentes..... 3
3 - Co-développement Humain-IA : stratégie et contributions..... 4
3.1 - Présentation de la stratégie..... 4
3.2 - Sources et supports de la stratégie : pourquoi cette approche est pertinente..... 4
4 - Co-développement Humain-IA : outils, activation et évaluation critique..... 5
4.1 - Outils IA utilisés et leur usage..... 5
4.2 - Comment les outils ont été activés, éléments de setup..... 5
4.3 - Évaluation critique..... 5
5 - Gestion de projet..... 6
5.1 - Méthodologie..... 6
5.2 - Suivi et contrôle qualité..... 6
5.3 - Bilan de gestion de projet..... 6

1 - Description du système : contexte, objectifs et fonctionnalités

1.1 - Contexte et objectif

Ce projet est un portfolio web full-stack développé dans le cadre d'un cours de développement full-stack à l'ENSIM. L'objectif principal est double : d'une part, produire un produit fonctionnel et déployable permettant à un développeur de présenter ses projets et compétences en ligne ; d'autre part, expérimenter une stratégie de co-développement humain-IA structurée, en utilisant Claude (Anthropic) comme agent de génération de code.

L'application répond à un besoin concret : disposer d'une vitrine numérique professionnelle avec une interface d'administration permettant de gérer le contenu dynamiquement, sans intervention dans le code source. Le périmètre a été défini avant tout développement via un document de spécification (), adopté comme source de vérité tout au long du projet.

1.2 - Fonctionnalités

Interface publique :

- Page d'accueil : section hero avec appels à l'action, carrousel des projets mis en avant (`featured`), aperçu des compétences principales.
- Page projets : liste complète des projets avec filtrage par catégorie et recherche textuelle.
- Page détail projets : description complète, galerie d'images, stack technique, liens GitHub et démo.
- Page compétences : compétences regroupées par catégorie (`frontend`, `backend`, `tools`, `language`, `other`) avec niveau de maîtrise (0–100).
- Page contact : formulaire de contact avec validation côté client, envoi d'e-mail via l'API Resend, et liens vers les réseaux sociaux (GitHub, LinkedIn).

Interface d'administration (protégée par JWT) :

- Login : authentification email/mot de passe avec génération de token JWT.
- Dashboard : statistiques (nombre de projets, projets en vedette, compétences) et liens rapides.
- Gestion des projets : CRUD complet avec gestion du statut `featured`, de l'ordre d'affichage et de l'upload d'images.
- Gestion des compétences : CRUD avec gestion des catégories et des niveaux de maîtrise.

Backend REST API :

- Endpoints `/api/auth`, `/api/projects`, `/api/skills`, `/api/contact`
- Accès public/protégé différencié par les middlewares `protect` et `adminOnly`
- Endpoint de santé `/api/health`
- Seed de l'administrateur initial via `/api/auth/seed`

1.3 - Expérience utilisateur

L'interface adopte un dark theme par défaut (palette "primary-900" via Tailwind CSS), cohérent avec les conventions du monde du développement. La navigation est responsive grâce à une navbar adaptative. Les composants UI réutilisables garantissent une cohérence visuelle sur l'ensemble des pages. Les transitions de route sont gérées par React Router v6, assurant une navigation fluide en Single Page Application. Côté performance, React Query gère le cache et l'état des données, évitant les rechargements inutiles et offrant une expérience réactive.

2 - Description du système : résultat, travaux restants et techniques full-stack

2.1 - Résultat final

Le portfolio est une application web full-stack complète et fonctionnelle :

Couche	Technologies	État
Frontend	React, TypeScript, Vite, Tailwind	Complet
State/Data	React Query (TanStack Query v5)	Complet
Backend	Node.js, Express, TypeScript	Complet
Base de données	MongoDB (via Docker Compose)	Complet
Authentification	JWT, bcrypt	Complet
Upload fichiers	Multer	Complet
Envoi d'e-mails	Resend API	Complet

L'application est démarrée localement via trois commandes : "docker compose up -d" (MongoDB), "npm run dev" dans "backend" et "npm run dev" dans "frontend". Le proxy Vite redirige "/api/*" vers "localhost:3000", éliminant les problèmes CORS en développement. L'ensemble du code est en TypeScript strict, avec ESLint configuré sur les deux projets.

2.2 - Ce qu'il reste à faire

- Tests automatisés : aucun test unitaire ni test d'intégration n'a été écrit. L'ajout de tests avec Jest (backend) et Vitest/Testing Library (frontend) serait la priorité pour passer en production.
- Déploiement cloud : le projet tourne localement via Docker Compose. Un déploiement réel (par exemple, frontend sur Vercel, backend sur Railway, MongoDB sur Atlas) n'a pas encore été réalisé.
- Sécurisation de la route "/seed" : actuellement publique, cette route devrait être désactivée ou sécurisée après la première utilisation en production.
- Gestion des erreurs côté client : les messages d'erreur de l'API peuvent être davantage contextualisés dans l'UI.
- Optimisation SEO : les balises `<meta>` et le rendu côté serveur (SSR) ne sont pas implémentés.

2.3 - Techniques full-stack pertinentes

Architecture REST avec séparation stricte des rôles : le backend expose une API REST stateless, le frontend consomme cette API via React Query. Cette séparation facilite les évolutions indépendantes des deux couches.

React Query pour la gestion d'état asynchrone : plutôt qu'un store global (Redux), React Query gère le cache, les états de chargement/erreur et l'invalidation automatique après mutation. Toutes les mutations appellent `queryClient.invalidateQueries` pour maintenir la cohérence de l'UI.

JWT avec intercepteur Axios : le token est stocké dans `localStorage` et injecté automatiquement dans chaque requête via un intercepteur Axios configuré dans `frontend/src/api/axios.ts`. Côté backend, le middleware `protect` vérifie la signature du token à chaque requête protégée.

Containerisation MongoDB via Docker Compose : l'infrastructure de base de données est reproductible et isolée, avec un volume persistant (`mongodb_data`). Cela garantit un environnement identique pour tous les développeurs.

Proxy Vite : en développement, les appels `/api/*` sont redirigés par le serveur de développement Vite vers `http://localhost:3000`, évitant la configuration CORS sans modifier le code de production.

Upload sécurisé avec Multer : les fichiers sont filtrés par type MIME, limités à 5 MB, et servis en statique depuis `backend/uploads/`. Ce dossier est exclu du versionnement Git.

3 - Co-développement Humain-IA : stratégie et contributions

3.1 - Présentation de la stratégie

La stratégie de co-développement adoptée repose sur un processus itératif en quatre phases, décrites ci-dessous :

1. Cadrage : définition du périmètre fonctionnel et technique, rédaction de la spécification (`SPEC.md`) et du fichier d'instructions pour Claude (`CLAUDE.md`), choix des technologies (React, Node.js, TypeScript, Docker). Cette phase est presque entièrement humaine : l'IA n'intervient pas dans les décisions architecturales, simplement dans la rédaction des fichiers markdown.
2. Génération : production du code par Claude via des prompts précis, structurés et itératifs. Chaque prompt décrit un module précis (ex. : « génère le contrôleur Express pour les projets avec les middlewares protect et adminOnly »). Claude Code (CLI) est utilisé directement dans l'éditeur.
3. Revue : relecture critique du code généré, correction des erreurs logiques, ajustements UX/design, vérification de la cohérence avec la spécification. L'humain (dans la limite du possible en fonction des connaissances des technologies) valide ou rejette chaque bloc de code produit.
4. Intégration : tests fonctionnels manuels, résolution des bugs d'intégration, assemblage des modules et démarrage via Docker Compose. Les bugs résiduels sont soumis à Claude pour analyse, mais la correction finale est validée par l'humain.

3.2 - Sources et supports de la stratégie : pourquoi cette approche est pertinente

Cette stratégie s'inspire du modèle spec-driven development : rédiger une spécification complète avant tout code permet de fournir à Claude un contexte stable et précis, réduisant la dérive entre les sessions. Le fichier CLAUDE.md joue le rôle de mémoire persistante inter-sessions, palliative à la limite principale des LLM (absence de mémoire entre conversations).

La pertinence de cette approche repose sur trois constats :

- Claude excelle dans la production de code boilerplate (setup Express, hooks React Query, schémas Mongoose) mais manque de vision globale sur l'architecture.
- La qualité du prompt détermine la qualité du résultat : des instructions vagues produisent du code générique ; des instructions précises (stack, contraintes, format attendu) produisent du code directement intégrable.
- L'itération rapide (prototype → revue → ajustement) s'est révélée plus productive qu'une spécification exhaustive suivie d'une génération monolithique.

L'IA a contribué à environ 98% du volume de code produit mais l'humain a assuré 100% des décisions architecturales, et de la validation fonctionnelle.

4 - Co-développement Humain-IA : outils, activation et évaluation critique

4.1 - Outils IA utilisés et leur usage

Claude Code (Anthropic) — outil principal

Claude Code est le CLI officiel d'Anthropic permettant d'interagir avec le modèle Claude directement depuis le terminal et l'éditeur de code (extension VS Code). Il a été utilisé pour :

- Générer l'intégralité du code backend (Express, Mongoose, middlewares JWT, Multer)
- Générer l'intégralité du code frontend (composants React, hooks React Query, pages publiques et admin)
- Analyser les erreurs TypeScript et proposer des corrections
- Rédiger et reformuler la documentation (README, SPEC.md, rapport)

Modèle utilisé : `claude-sonnet-4-6` (Claude Sonnet 4.6)

4.2 - Comment les outils ont été activés, éléments de setup

Installation et configuration de Claude Code :

"npm install -g @anthropic-ai/claude-code"

"claude" (démarrage du CLI interactif)

Fichier CLAUDE.md : ce fichier est lu automatiquement par Claude Code au démarrage de chaque session. Il contient :

- Le contexte du projet et la stack technique
- Les commandes essentielles (démarrage, build, lint)
- L'architecture et les conventions à respecter
- Les pièges connus (ex. : le script `kill-port 3000` avant le démarrage du backend)

Ce fichier joue un rôle central : il compense l'absence de mémoire inter-sessions du modèle en fournissant un contexte standardisé à chaque nouvelle conversation. Sans lui, chaque session nécessiterait de ré-expliquer l'ensemble du projet.

Extension VS Code : Claude Code est intégré dans l'éditeur, permettant de soumettre des portions de code sélectionnées directement au modèle et d'appliquer les modifications suggérées en un clic.

Bibliothèque de prompts : une collection de prompts types a été construite au fil du projet pour les tâches récurrentes (génération d'un controller Express, génération d'un hook React Query, correction d'une erreur TypeScript). Ces prompts réutilisables ont amélioré la cohérence des résultats.

4.3 - Évaluation critique

Dimension quantitative :

- ~98% du code produit avec assistance IA
- 4 phases de travail structurées
- ~20 commits sur la durée du projet, couvrant backend, frontend, infrastructure et documentation
- Application full-stack complète livrée : 5 pages publiques, 4 pages admin, 15+ endpoints REST
- Temps de développement estimé significativement réduit par rapport à un développement traditionnel solo

Dimension qualitative :

Points forts :

- La génération de code boilerplate (setup Express, schémas Mongoose, hooks React Query) est quasi-instantanée et de bonne qualité.
- Claude respecte les conventions TypeScript strict et les patterns architecturaux décrits dans CLAUDE.md.
- La productivité sur les tâches répétitives (CRUD, forms, routes) est nettement supérieure.

Limites :

- Cohérence inter-sessions : chaque session repart de zéro. Sans CLAUDE.md, le modèle génère du code en contradiction avec le reste de la codebase.
- "Sur-génération" : Claude tend à produire plus que nécessaire (gestion d'erreurs superflue, abstractions prématurées, commentaires inutiles). Une étape de simplification humaine est systématiquement nécessaire.
- "Debugging non-trivial" : les erreurs dans le code généré sont parfois subtiles (mauvais typage TypeScript, ordre de middleware incorrect). La localisation du bug nécessite une expertise humaine.
- "Dépendance au prompt" : un prompt imprécis produit du code générique. La maîtrise de l'art du prompting est une compétence à part entière, sous-estimée au départ.

Conclusion critique : le co-développement humain-IA est efficace lorsque les rôles sont clairement définis : l'IA comme outil de production, l'humain comme architecte et décideur. Cette expérience démontre que la valeur ajoutée du développeur ne disparaît pas avec l'IA : elle se déplace vers la conception, la validation et l'orchestration.

5 - Gestion de projet

5.1 - Méthodologie

Le projet a adopté une approche itérative et incrémentale, proche des principes agiles, organisée en sprints informels correspondant aux 4 phases de la stratégie de co-développement. L'absence d'une équipe (projet solo) a simplifié la coordination mais exigé une discipline personnelle pour respecter la séquence Cadrage → Génération → Revue → Intégration à chaque incrément.

5.2 - Suivi et contrôle qualité

- Versionning Git : chaque fonctionnalité majeure fait l'objet d'un commit conventionnel (`feat:`, `fix:`, `chore:`, `docs:`). L'historique Git constitue le journal de bord du projet.
- Tests manuels : validation fonctionnelle systématique après chaque intégration (démarrage du serveur, test des endpoints via curl/navigateur, vérification de l'UI).
- Revue du code généré : chaque bloc de code produit par Claude est relu avant intégration. Les ajouts superflus sont supprimés, les types `any` sont éliminés, les conventions CLAUDE.md sont vérifiées.
- Absence de tests automatisés : identifiée comme dette technique à traiter en priorité pour une mise en production réelle.

5.3 - Bilan de gestion de projet

Le projet a été livré dans les délais avec l'ensemble des fonctionnalités spécifiées. La stratégie de co-développement a permis de compenser le contexte de développeur solo en réduisant considérablement le

temps de production du code. Les principaux facteurs de succès sont : la rédaction préalable d'une spécification complète, l'utilisation de CLAUDE.md comme mémoire persistante, et la discipline de revue systématique du code généré. Le principal facteur de risque identifié reste la dépendance à la qualité du prompt et l'absence de tests automatisés pour fiabiliser les évolutions futures.